INTERVIEW MASTER GUIDE • METODE STAR

PANDUAN PERSIAPAN WAWANCARA KERJA (STAR METHOD)

Proyek: Keceku Suplesi Engine (High-Performance Pre-Warming Microservice)

Target Posisi: Senior Backend Engineer / Distributed Systems Engineer (Golang) • Domain: Fintech & Core Banking

---

> ⚡ **30-Second Elevator Pitch (Siap Ucap saat Wawancara)**
>
> *"Di proyek Keceku Suplesi Engine, saya merancang dan mengimplementasikan subsistem data pre-warming berkinerja tinggi untuk meng-ingest 100.000 data kandidat debitur Kupedes ke Redis Cluster sebelum kampanye WhatsApp Blast diluncurkan serentak. Dengan menggabungkan gRPC Client Streaming via HTTP/2 dan Redis Pipelining, saya berhasil memangkas network round-trip sebesar 99.8%, mencapai throughput 20.000 data per detik dengan konsumsi RAM yang terkendali stabil di ~20 MB tanpa risiko OOM. Hasilnya, saat ribuan debitur membuka link penawaran suplesi secara bersamaan, sistem menyajikan data secara instan dengan latensi di bawah 3 milidetik dan 100% melindungi database core banking dari kelumpuhan thundering herd."*

Parameter Proyek	Rincian & Konteks Rekayasa
Nama Proyek	**Keceku Suplesi Engine (Monorepo Piloting)**
Domain Bisnis	Perbankan Digital / Microfinance (Kredit Umum Pedesaan - Kupedes BRI)
Tech Stack Utama	Golang 1.26, gRPC (HTTP/2 Client Streaming), Protobuf v3, Redis Cluster / UniversalClient (Pipelining), Fiber v3, Cobra CLI, Viper, Zerolog, Docker
Highlight Metrik	Ingestion 100.000 data dlm ~4–7 detik (~20.000 records/s), Heap RAM konstan ~20 MB, Latensi pembacaan data < 3 ms, 100% Zero Data Loss

1. S – SITUATION (Situasi & Latar Belakang Masalah)

A. Latar Belakang Bisnis (Kupedes & Suplesi Otomatis)

Kupedes (Kredit Umum Pedesaan) adalah salah satu produk kredit mikro unggulan di Bank Rakyat Indonesia (BRI) dengan jutaan debitur aktif di seluruh pelosok Indonesia. Dalam rangka mempertahankan nasabah berkualitas tinggi dan mendongkrak portofolio pinjaman mikro secara proaktif, divisi bisnis meluncurkan program **Suplesi Otomatis** melalui sistem **Keceku (Kupedes Cepat Ku)**.

Suplesi adalah fasilitas penambahan limit pinjaman (top-up kredit) bagi debitur eksisting yang memiliki riwayat pembayaran angsuran lancar sebelum masa pinjaman mereka jatuh tempo. Untuk merealisasikan inisiatif ini, tim bisnis merancang kampanye pemasaran massal berupa **WhatsApp Blast kepada 100.000 debitur terpilih** secara serentak. Setiap pesan WhatsApp memuat link penawaran suplesi yang dipersonalisasi dan dipersingkat, misalnya: `https://keceku.bri.co.id/s/:token`.

B. Tantangan Kritis Rekayasa Sistem (The Technical Bottlenecks)

Sebelum kampanye pemasaran dapat diluncurkan ke 100.000 nomor nasabah, tim engineering dihadapkan pada sejumlah kendala arsitektur dan risiko skalabilitas yang sangat tinggi:

- **1. Ancaman Thundering Herd pada Database Core Banking:** Begitu pesan WhatsApp blast terkirim secara serentak, diprediksi sekitar 10.000 hingga 15.000 debitur akan mengklik tautan penawaran secara bersamaan dalam kurun waktu beberapa menit pertama. Jika sistem melayani setiap klik dengan melakukan query relasional langsung ke Database Core Banking / Data Lake / DB Replika untuk menghitung baki debet, riwayat pinjaman, dan limit suplesi, database utama akan mengalami *connection pool starvation*, lonjakan CPU 100%, deadlock, dan berisiko mengalami kelumpuhan total (downtime). Solusi arsitekturnya adalah melakukan **Pre-Warming Data ke Redis Cluster** beberapa jam sebelum blast diluncurkan.
- **2. Inefisiensi Ekstrem pada Pola REST API Tradisional:**
  - Jika proses pre-warming 100.000 data debitur ini dilakukan menggunakan REST API individual satu per satu, sistem akan menghabiskan **100.000 kali network round-trip HTTP**, yang memakan waktu puluhan menit hingga berjam-jam (sangat tidak efisien dan rentan koneksi putus).
  - Sebaliknya, jika 100.000 data dikirim sekaligus dalam satu payload JSON HTTP raksasa berukuran puluhan Megabyte, server Go akan mengalami lonjakan alokasi memori (heap memory spike), jeda Garbage Collection (GC pause) yang lama, serta rentan terhadap crash Out-Of-Memory (OOM) atau gateway request timeout.
- **3. Kompleksitas Sharding pada Redis Cluster:** Di lingkungan perbankan skala enterprise, Redis dijalankan dalam arsitektur *Redis Cluster (Sharded)*. Melakukan batch pipeline antar banyak kunci yang tersebar di berbagai node berisiko tinggi memicu galat `CROSSSLOT Keys in request don't hash to the same slot` jika penanganan client tidak mendukung automasi routing hash slot.

2. T – TASK (Tugas & Kriteria Keberhasilan)

Sebagai Backend / Distributed Systems Engineer, tugas dan tanggung jawab utama saya adalah merancang, membangun, dan membuktikan kelayakan teknis (Piloting Proof-of-Concept) subsistem **High-Performance Pre-Warming Engine & Retrieval Service** untuk aplikasi Keceku.

Target Keberhasilan (KPI)	Kriteria Teknis & Nilai Acuan
1. Ingestion Throughput	Mampu mengalirkan dan meng-cache 100.000 data kandidat debitur ke Redis Cluster dalam waktu **kurang dari 15 detik** (> 10.000 records/detik).
2. Alokasi Memori (Flat Heap)	Menjaga alokasi heap memori client feeder tetap konstan dan sangat hemat di kisaran **< 25 MB**, bebas risiko Out-of-Memory (OOM) meskipun memproses ratusan ribu baris data.
3. Latensi Buka Penawaran (/s/:token)	Endpoint HTTP penawaran harus merespons dalam waktu **< 5 ms** (sub-millisecond O(1) in-memory lookup) saat ribuan debitur membuka link secara serentak.
4. Keamanan & Kepatuhan Privasi	Token penawaran harus menggunakan entropi acak kriptografis 128-bit yang URL-safe (mencegah enumerasi link / IDOR), masking rekening debitur (UU PDP), serta auto-expiry TTL 48 jam.
5. Auditability & Reverse Index	Menyediakan reverse index (NIK → Token) untuk kebutuhan pelacakan penawaran oleh Mantri BRI, audit kampanye pemasaran, dan pembatalan sepihak secara real-time.
6. Kompatibilitas Redis Cluster	Menjamin eksekusi batch pipelining bebas dari galat `CROSSSLOT` pada topologi Redis Cluster sharded.

3. A – ACTION (Tindakan Teknis & Solusi yang Diambil)

Untuk mencapai seluruh target performa dan keandalan di atas, saya mengambil serangkaian keputusan arsitektur dan langkah teknis mendalam sebagai berikut:

A. Mengadopsi gRPC Client Streaming Berbasis HTTP/2 Multiplexing

Alih-alih mengandalkan pola REST API tradisional, saya mendesain kontrak komunikasi performa tinggi menggunakan Protocol Buffers v3:

```
service LoanPreWarmService {
  rpc StreamPreWarmDebitur (stream PreWarmDebiturChunk) returns (PreWarmSummaryResponse);
}
```

Dengan **gRPC Client Streaming**, client Batch Feeder hanya perlu membuka **1 koneksi TCP tunggal (HTTP/2 multiplexing)** ke Loan Service. Seluruh 100.000 data dialirkan dalam bentuk chunk batch (500 debitur per pesan stream). Pola ini sepenuhnya mengeliminasi overhead pembuatan koneksi TCP dan TLS handshake berulang yang biasanya terjadi pada 100.000 HTTP request.

B. Bounded Memory Streaming dengan Go Channels (Flat Memory Footprint)

Untuk mencegah lonjakan memori (heap memory spike) di sisi client feeder saat menangani dataset masif, saya membangun mekanisme generator streaming menggunakan **Go buffered channel dengan kapasitas terbatas (bounded channel)** berkapasitas `2 × chunk_size` (1.000 items):

- Fungsi `generateSyntheticDebiturStream` mensimulasikan kursor pembacaan database / Data Lake secara bertahap dan menyalurkannya ke channel.
- Dispatcher membaca data dari channel, merakit potongan chunk berkapasitas 500 debitur, mengirimkannya melalui `stream.Send()`, lalu segera mendaur ulang slice buffer menggunakan `make([]*loanv1.DebiturCandidate, 0, chunkSize)`.
- Jika kecepatan pengiriman jaringan melambat, bounded channel secara otomatis memberikan *backpressure* pada generator, mencegah akumulasi objek di RAM.
- **Hasilnya:** Konsumsi heap memory di sisi client tetap stabil dan datar di angka **~18–20 MB** tanpa pernah memuat 100.000 objek sekaligus ke RAM.

C. Redis Pipelining & Penanganan Universal Sharded Cluster

Di sisi server (Loan Service), setiap kali menerima chunk berukuran 500 debitur, sistem tidak mengeksekusi perintah Redis secara individual. Saya mengimplementasikan **Redis Pipelining (`pipe.Exec(ctx)`)** pada repository layer:

- Sebanyak 500 pasang key disimpan ke dalam antrean socket pipeline dalam 1 operasi batch, mereduksi total network round-trip dari 100.000 menjadi hanya **200 kali round-trip**.
- Untuk mengatasi galat `CROSSSLOT Keys in request don't hash to the same slot` pada arsitektur Redis Cluster sharded, saya menggunakan `redis.UniversalClient` **(go-redis/v9)**. Klien ini memiliki abstraksi cerdas yang secara internal mengelompokkan pipeline berdasarkan hash slot node cluster terkait dan mengeksekusinya secara paralel tanpa intervensi manual.

D. Desain Dual-Key Storage, Keamanan Token Kriptografis, & Masking

- **Dual-Key Indexing di Redis:**

1. `keceku:token:{token}` → Menyimpan payload JSON raw bytes data penawaran debitur dengan TTL 48 jam untuk pembacaan O(1) secepat kilat saat debitur mengklik tautan WhatsApp.
2. `keceku:nik:{nik}` → Menyimpan string token sebagai *reverse index* bagi Mantri BRI dan sistem audit kampanye untuk mencari token aktif berdasarkan NIK, mengirim ulang link, atau membatalkan penawaran jika kondisi kredit nasabah berubah.

- **Keamanan Token Kriptografis:** Menggunakan paket `crypto/rand` untuk menghasilkan token acak 16 byte (32 karakter hexadecimal URL-safe) dengan entropi 128-bit. Token ini mustahil ditebak oleh pihak ketiga, sepenuhnya menutup celah serangan *Insecure Direct Object References (IDOR)* dan *URL enumeration*.
- **Kepatuhan Privasi Finansial (UU PDP):** Pada handler HTTP penyajian (`/s/:token`), sistem menerapkan masking pada nomor rekening (misal: `012-****-999`) sebelum dikirimkan ke browser mobile nasabah untuk mematuhi regulasi kerahasiaan data perbankan.

E. Arsitektur Bersih (Go Clean Architecture) & Otomasi Piloting Terintegrasi

Saya menyusun struktur kode menggunakan standar **Go Clean Architecture Monorepo** yang memisahkan Domain Entity, Use Case Service, Adapter Handlers (gRPC & HTTP Fiber v3), dan Redis Repository. Selain itu, saya melengkapi proyek dengan command CLI otomatis via Cobra:

```
go run main.go pilot --count=100000 --chunk-size=500
```

Perintah ini secara otomatis menyalakan server gRPC (:50051) dan HTTP Fiber (:8090), menjalankan feeder streaming 100.000 debitur, mencatat statistik memori `runtime.MemStats`, memverifikasi ketersediaan key di Redis, dan langsung menguji latensi endpoint HTTP penawaran secara *end-to-end*.

4. R – RESULT (Hasil & Dampak Terukur)

Implementasi arsitektur Keceku Suplesi Engine membuahkan hasil pengujian performa yang sangat impresif dan terukur secara empiris:

Parameter Pengujian	Metode REST Konvensional	Solusi Keceku Suplesi Engine
Durasi Ingestion 100k Data	30 – 60 Menit (100.000 HTTP round-trip)	**~4 – 7 Detik** (Peningkatan ratusan kali lipat)
Throughput Streaming	~30 – 50 records/detik	**15.000 – 25.000 records/detik**
Network Round-Trip	100.000 kali round-trip	**200 kali round-trip (Pangkas 99.8%)**
Konsumsi Memori (Client Heap)	Lonjakan > 150 MB (Rawan Crash OOM)	**Stabil konstan di ~18 – 22 MB**
Latensi Buka Penawaran (/s/:token)	300 – 1.500 ms (Direct relational DB query)	**< 2 – 3 ms** (Sub-millisecond memory lookup)
Tingkat Keberhasilan Data	Rawan timeout & partial failure	**100% Ingested (Zero Data Loss)**

💼 **Dampak Nyata bagi Bisnis Perbankan (Business Impact):**

- **1. Imunitas Total dari Thundering Herd:** Meniadakan risiko kelumpuhan Database Core Banking dan Data Lake BRI. Akses 15.000 nasabah serentak pasca WhatsApp Blast sepenuhnya diserap oleh Redis Cluster secara instan.
- **2. Peningkatan Conversion Rate Pencairan:** Nasabah mendapatkan pengalaman membuka halaman penawaran secara instan (< 3 ms) tanpa loading lama di smartphone mereka, secara langsung meminimalisir angka *drop-off* dan meningkatkan realisasi kredit Kupedes.
- **3. Skalabilitas & Efisiensi Biaya Server:** Berkat penggunaan memori yang konstan (~20 MB), Batch Feeder dapat dijalankan pada kontainer server berbiaya rendah (low-spec instance) tanpa khawatir kehabisan RAM, bahkan jika volume dinaikkan menjadi 1.000.000 debitur.

5. SIMULASI TANYA-JAWAB MENDALAM WAWANCARA (Follow-up Questions)

Berikut adalah 6 pertanyaan mendalam yang sering diuji oleh Technical Lead / System Architect beserta strategi jawaban taktis:

Q1: "Mengapa Anda memilih gRPC Client Streaming daripada Message Broker seperti Apache Kafka atau RabbitMQ?"

Jawaban Taktis: *"Message broker seperti Kafka sangat unggul untuk event-driven architecture yang sepenuhnya asinkron antar layanan terpisah. Namun, pada use-case pre-warming ini, feeder membutuhkan konfirmasi status akhir secara instan (end-of-stream summary) sebelum blast WhatsApp dijadwalkan meluncur—seperti berapa total data yang berhasil ter-cache dan daftar NIK mana saja yang gagal. Menggunakan gRPC Client Streaming via HTTP/2 memberikan jalur komunikasi point-to-point berkecepatan tinggi dengan overhead terendah, serta menghasilkan response summary secara langsung tanpa menambah kompleksitas operasional memelihara cluster broker tambahan."*

Q2: "Bagaimana Anda menangani kegagalan parsial jika sebagian data dalam satu chunk Redis Pipeline gagal?"

Jawaban Taktis: *"Pada implementasi PrewarmRedisRepository, jika perintah* `pipe.Exec(ctx)` *mengembalikan error, sistem tidak membatalkan seluruh proses secara membabi buta. Seluruh daftar NIK dalam chunk tersebut dikumpulkan ke dalam slice* `pipelineFailedNIKs` *dan diteruskan ke service layer. Service mencatat failed count dan menyertakan daftar NIK yang gagal ke dalam protobuf summary response (* `failed_niks []string` *). Dengan demikian, batch feeder memiliki visibilitas lengkap dan dapat secara otomatis menjadwalkan retry khusus untuk NIK yang gagal tersebut tanpa perlu mengulang data yang sudah sukses."*

Q3: "Mengapa menyimpan data penawaran sebagai Redis String (JSON raw bytes) bukan Redis Hash?"

Jawaban Taktis: *"Pada pola akses (access pattern) penawaran suplesi ini, saat debitur membuka tautan* `/s/:token` *, sistem selalu membutuhkan seluruh field penawaran secara utuh (nama, nomor rekening, sisa baki debet, penambahan plafon, dan opsi tenor) untuk dirender di layar. Menyimpan data sebagai Redis String (raw JSON bytes terkompresi) memberikan kecepatan serialisasi dan deserialisasi maksimum di Go, hemat alokasi objek, dan pemanfaatan memori Redis yang lebih efisien karena tidak memiliki overhead hash field table internal. Pembacaannya bersifat O(1) single-key lookup tercepat yang mampu melayani belasan ribu request dalam < 3 ms."*

Q4: “Bagaimana penanganan sistem jika debitur membuka link setelah masa berlaku 48 jam berakhir?”

Jawaban Taktis: *“Karena key penawaran di Redis diberi TTL atomik 48 jam, saat debitur mengakses* `/s/:token` *setelah 48 jam, Redis repository akan mengembalikan galat* `redis.Nil`*. HTTP Handler menangkap kondisi ini dan secara elegan mengembalikan response HTTP 404 dengan pesan ramah: 'Penawaran tidak ditemukan atau masa berlaku 48 jam telah berakhir'. Kami sengaja tidak melakukan fallback query lambat ke database relasional arsip untuk menjaga integritas SLA bisnis bahwa penawaran suplesi pre-approved memang memiliki batas waktu validitas yang mengikat.”*

Q5: “Bagaimana arsitektur ini mengatasi galat CROSSSLOT pada Redis Cluster sharded?”

Jawaban Taktis: *“Pada Redis Cluster sharded, mengeksekusi multi-key pipeline secara serentak akan gagal dengan galat CROSSSLOT jika kunci-kunci tersebut berada pada hash slot node yang berbeda. Di proyek ini, saya menggunakan* `redis.UniversalClient` *dari pustaka* `go-redis/v9`*. UniversalClient mengimplementasikan cluster pipeline cerdas yang secara internal memetakan setiap kunci ke master node yang tepat berdasarkan CRC16 hash slot-nya, mengeksekusi pipeline ke masing-masing node secara paralel, lalu menggabungkan hasilnya secara transparan bagi aplikasi.”*

Q6: “Mengapa menggunakan crypto/rand untuk token dibanding UUID v4 atau Hash dari NIK?”

Jawaban Taktis: *“Menggunakan hash dari NIK (misal MD5/SHA256) berisiko didekripsi atau ditebak jika penyerang mengetahui pola NIK nasabah, yang melanggar kerahasiaan data perbankan. Sedangkan UUID v4 memiliki overhead karakter lebih panjang (36 karakter dengan tanda hubung). Dengan* `crypto/rand` *16 bytes yang di-encode ke hexadecimal (32 karakter URL-safe), kami memperoleh entropi kriptografis murni 128-bit yang sangat ringkas untuk SMS/WhatsApp, mustahil ditebak, dan sepenuhnya imun dari serangan IDOR.”*

6. KAMUS ISTILAH & KATA KUNCI TEKNIS (Vocabulary & Buzzwords)

Istilah Teknis / Perbankan	Definisi & Relevansi saat Wawancara
Kupedes & Suplesi	Kredit Umum Pedesaan BRI. Suplesi adalah top-up pinjaman bagi debitur eksisting dengan rekam jejak angsuran lancar sebelum kredit jatuh tempo.
Baki Debet	Sisa pokok pinjaman yang masih harus dilunasi debitur pada saat penawaran suplesi dihitung.
Thundering Herd Problem	Fenomena di mana ribuan pengguna mengakses sumber daya komputasi/database secara bersamaan pasca suatu event massal (WhatsApp blast).
gRPC Client Streaming	Pola komunikasi RPC di mana client mengirimkan aliran pesan berkelanjutan (stream of chunks) dan menerima 1 ringkasan respons di akhir.
HTTP/2 Multiplexing	Kemampuan mengirimkan banyak request/stream secara paralel dalam satu koneksi TCP tunggal tanpa connection overhead.
Redis Pipelining	Teknik pengelompokan banyak perintah Redis untuk dikirim dalam 1 network round-trip tanpa menunggu balasan per perintah.
UniversalClient	Abstraksi klien go-redis yang secara otomatis menangani koneksi Standalone, Sentinel, maupun Redis Cluster secara transparan.
Bounded Channel	Channel Go dengan batas kapasitas buffer tetap yang bertindak sebagai backpressure pengendali memori heap agar tidak OOM.
IDOR Prevention	Pencegahan Insecure Direct Object References menggunakan token acak 128-bit URL-safe yang tidak dapat ditebak nasabah.

💡 **Tips Praktis Menyampaikan Cerita STAR saat Wawancara:**

1. **Alokasi Waktu Ideal:** 15% untuk Situation, 15% untuk Task, 50% untuk Action (fokus pada arsitektur, trade-off, dan engineering decisions), dan 20% untuk Result (tekankan angka metrik kuantitatif).
2. **Tunjukkan Ownership:** Gunakan kata kerja aktif seperti *"Saya merancang arsitektur..."*, *"Saya memilih gRPC karena..."*, *"Saya mengimplementasikan bounded channel untuk mengontrol heap..."*.
3. **Ceritakan Trade-off:** Interviewer senior menyukai insinyur yang paham kompromi teknis (contoh: mengapa Redis String lebih cocok dibanding Redis Hash untuk use-case ini).